Hyderabad Karnataka Education Society's

POOJYA DODDAPPA APPA COLLEGE OF ENGINEERING KALABURAGI

(AUTONOMOUS INSTITUTION)

DEPARTMENT OF MATHEMATICS



LAB MANUAL OF MATHEMATICS FOR II SEMESTER CSE STREAM USING PYTHON PROGRAMMING

Compiled by:

Dr. A. G. Hiremath.
Professor and Head

Prof. Ravi.S.kambar.
Assistant Professor.

Prof. Sangeeta.V.pattanashetty.
Assistant Professor.

Course: Numerical Methods.

Course Code: CSMAT201.

List of Laboratory Experiments:

1) Errors and approximation,
2) Roots finding methods,
3) Norms, Condition number,
4) Gauss Seidel method and Rayleigh power's method,
5) Forward and Backward interpolation,
6) Lagrange's interpolation,
7) Numerical interpolation,
8) Taylor's method, Modified Euler's method,
9) Runge-Kutta method of fourth order,
10) Finding Gradient, divergence and curl.

LAB-1

1. Errors and approximation

I. Find the absolute error, relative error, and percentage error for π

```
from math import *
true, approx = pi, 22/7
print("Absolute Error:", abs(true-approx))
print("Relative Error:", abs(true-approx)/true)
print("Percent Error:", abs(true-approx)/true*100)
```

```
Absolute Error: 0.0012644892673496777
Relative Error: 0.0004024994347707008
Percent Error: 0.04024994347707008
```

II. Find the absolute error, relative error, and percentage error of e^x for $x = 1$

```
from math import *
x = 1
true, approx = exp(x), 1+x+x**2/factorial(2)
print("Absolute Error:", abs(true-approx))
print("Relative Error:", abs(true-approx)/true)
print("Percent Error:", abs(true-approx)/true*100)
```

```
Absolute Error: 0.2182818284590451
Relative Error: 0.08030139707139415
Percent Error: 8.030139707139416
```

III. Find the absolute error, relative error, and percentage error of $\sin(x)$ for $x = \pi$.

```
from math import *
x = pi
true, approx = sin(x), x-x**3/factorial(3)
print("Absolute Error:", abs(true-approx))
print("Relative Error:", abs(true-approx)/true)
print("Percent Error:", abs(true-approx)/true*100)
```

```
Absolute Error: 2.0261201264601763
Relative Error: 1.6544526371773804e+16
Percent Error: 1.6544526371773804e+18
```

Exercise

1 . Find the absolute error, relative error, and percentage error of e^{2x} for x=3.
2 . Find the absolute error, relative error, and percentage error of *tanx* for x=π.
3 . Find the absolute error, relative error, and percentage error of *secx* for x=2π.

LAB-2

Roots finding methods,

1. Regula-Falsi Method

I. Obtain a root of the equation $x^3 - 2x - 5 = 0$ between 2 and 3 by Regula-falsi method. Perform 8 iterations.

```
from sympy import *
x = Symbol('x')
g = input('Enter the function: ')
f = lambdify(x, sympify(g))
a = float(input('Enter a value: '))
b = float(input('Enter b value; '))
N = int(input('Enter number of iteration; '))
print('Iteration \t Root \t Function value \n')
for i in range(1, N+1):
    c = (a*f(b)-b*f(a))/(f(b)-f(a))
    if (f(a)*f(c)<0):
        b=c
    else:
        a=c
        print('%d \t\t %0.4f \t %0.4f'% (i, c, f(c)))
print("The root of the equation is %0.4f" %c)
```

```
Enter the function: x**3-2*x-5
Enter a value: 2
Enter b value; 3
Enter number of iteration; 8
Iteration        Root    Function value

1                2.0588          -0.3908
2                2.0813          -0.1472
3                2.0896          -0.0547
4                2.0927          -0.0202
5                2.0939          -0.0075
6                2.0943          -0.0027
7                2.0945          -0.0010
8                2.0945          -0.0004
The root of the equation is 2.0945
```

2. Newton – Raphson Method

I. Find a root of the equation $3x = cosx + 1$, near 1, by Newton-Raphson method. Perform 5 iterations.

```
from sympy import *
x = Symbol('x')
g = input('Enter the function ')
f = lambdify(x,sympify(g))
dg = diff(g,x)
df = lambdify(x,sympify(dg))
x0 = float(input('Enter the initial approximation '))
n = int(input('Enter the number of iterations '))
for i in range(1, n+1):
    x1 = x0- (f(x0)/df(x0))
    print('%d \t\t %0.4f \t %0.4f \n' %(i, x1, f(x1)))
    x0 = x1
print(" The root of the equation is %0.4f" %x1)
```

```
Enter the function 3*x-cos(x)-1
Enter the initial approximation 1
Enter the number of iterations 5
1               0.6200          0.0462

2               0.6071          0.0001

3               0.6071          0.0000

4               0.6071          0.0000

5               0.6071          0.0000

 The root of the equation is 0.6071
```

Exercise

1 . Find a root of the equation $3x=cosx+1$, between 0and 1 , by Regula- falsi method.

2 . Find a root of the equation $xe^x=2$, between 0and 1 , by Regula- falsi method.

3. Obtain a real positive root of $x^4-x=0$, near 1, by Newton – Raphson method. Perform 4 iteration.

4. Obtain a real positive root of $x^4+x^3-7x^2-x+5=0$, near 3, by Newton – Raphson method.

LAB – 3

Norms, Condition number,

1. Norms Number

I. Find the norm for the vector (3, -4, 1) and the matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix}$

```
from numpy import *
v = array([3,-4,1])
A = array([[1,2,3],[4,5,6],[7,8,10]])
print("Vector norms:")
print("L1 =", linalg.norm(v,1))
print("L2 =", linalg.norm(v,2))
print("Inf =",linalg.norm(v,inf))
print("Matrix norms:")
print("Frobenius norm =", linalg.norm(A,'fro'))
print("1-norm =", linalg.norm(A,1))
print("Inf-norm =",linalg.norm(A,inf))
```

```
Vector norms:
L1 = 8.0
L2 = 5.0990195135927845
Inf = 4.0
Matrix norms:
Frobenius norm = 17.435595774162696
1-norm = 19.0
Inf-norm = 25.0
```

II . Find the norm for the vector (2,-4,2) and the matrix $\begin{bmatrix} 3 & 1 & 3 \\ 8 & 7 & 6 \\ 7 & 0 & 10 \end{bmatrix}$

```
from numpy import *
v = array([2,-4,2])
A = array([[3,1,3],[8,7,6],[7,0,10]])
print("Vector norms:")
print("L1 =", linalg.norm(v,1))
print("L2 =", linalg.norm(v,2))
print("Inf =",linalg.norm(v,inf))
print("Matrix norms:")
print("Frobenius norm =", linalg.norm(A,'fro'))
print("1-norm =", linalg.norm(A,1))
print("Inf-norm =",linalg.norm(A,inf))
```

```
Vector norms:
L1 = 8.0
L2 = 4.898979485566356
Inf = 4.0
Matrix norms:
Frobenius norm = 17.804493814764857
1-norm = 19.0
Inf-norm = 21.0
```

2.Condition Number

I. Find the condition number for the matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix}$

```
from numpy import *
A = array([[1,2,3],[4,5,6],[7,8,10]])
print("\nCondition numbers:")
print("cond_2    =", linalg.cond(A,2))
print("cond_1    =", linalg.cond(A,1))
print("cond_inf  =", linalg.cond(A,inf))
```

```
Condition numbers:
cond_2    = 88.4482799206987
cond_1    = 132.99999999999997
cond_inf  = 158.33333333333331
```

II. Find the condition number for the matrix $\begin{bmatrix} 3 & 1 & 3 \\ 8 & 7 & 6 \\ 7 & 0 & 10 \end{bmatrix}$

```
from numpy import *
A = array([[3,1,3],[8,7,6],[7,0,10]])
print("\nCondition numbers:")
print("cond_2 =", linalg.cond(A,2))
print("cond_1 =", linalg.cond(A,1))
print("cond_inf =", linalg.cond(A,inf))
```

```
Condition numbers:
cond_2 = 65.285503308591
cond_1 = 119.31999999999995
cond_inf = 79.79999999999998
```

Exercise

1 . Find the norm for the vector (1, 3, 8) and the matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix}$

2 . Find the norm for the vector (7, 12, 6) and the matrix $\begin{bmatrix} 17 & 7 & 2 \\ 0 & 5 & 4 \\ 7 & 5 & 1 \end{bmatrix}$

3 . Find the condition number for the matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix}$

4. Find the condition number for the matrix $\begin{bmatrix} 17 & 7 & 2 \\ 0 & 5 & 4 \\ 7 & 5 & 1 \end{bmatrix}$

LAB-4

Gauss Seidel method and Rayleigh power's method.

1. Gauss- seidel Method

I. Solve the following system of equation using the Gauss-Seidel method

$$\mathbf{20}x + y - \mathbf{2}z = \mathbf{17},\ \mathbf{3}x + \mathbf{20}y - z = -\mathbf{18},\ \mathbf{2}x - \mathbf{3}y + \mathbf{20}z = \mathbf{25}.$$

```
f1 = lambda x,y,z:(17-y+2*z)/20
f2 = lambda x,y,z:(-18-3*x+z)/20
f3 = lambda x,y,z:(25-2*x+3*y)/20
x0, y0, z0 = 0, 0, 0
e = float(input('Enter tolerable error : '))
print('\t Iteration \t x \t\t y \t\t z \n')
for i in range (0, 25):
    x1 = f1(x0, y0, z0)
    y1 = f2(x1, y0, z0)
    z1 = f3(x1, y1, z0)
    print('\t %d \t\t %0.4f \t %0.4f \t %0.4f\n' %(i, x1, y1, z1))
    e1, e2, e3 = abs(x0-x1), abs(y0-y1), abs(z0-z1)
    x0,y0,z0 = x1,y1,z1
    if e1>e and e2>e and e3>e:
        continue
    else :
        break
print('\n Solution : x = %0.3f , y = %0.3f and z = %0.3f\n'% (x1, y1, z1))
```

```
Enter tolerable error : 0.001
         Iteration       x              y              z

         0               0.8500         -1.0275        1.0109

         1               1.0025         -0.9998        0.9998

         2               1.0000         -1.0000        1.0000


 Solution : x = 1.000 , y = -1.000 and z = 1.000
```

II. Solve the following system of equation using the Gauss-Seidel method

$$x + 2y - z = 3,\ 3x - y + 2z = 1, 2x - 2y + 6z = 2.$$

```
f1 = lambda x, y, z: (1+y-2*z)/3
f2 = lambda x, y, z: (3-x+z)/2
f3 = lambda x, y, z: (2-2*x+2*y)/6
x0, y0, z0 = 0, 0, 0
e = float(input('Enter tolerated error : '))
print('\t Iteration \t x \t\t y \t\t z \n')
for i in range(0, 25):
    x1 = f1(x0, y0, z0)
    y1 = f2(x1, y0, z0)
    z1 = f3(x1, y1, z0)
    print('\t %d \t\t %.4f \t %0.4f \t %0.4f \n' %(i, x1, y1, z1))
    e1,e2,e3 = abs(x0-x1), abs(y0-y1), abs(z0-z1)
    x0,y0,z0 = x1,y1,z1
    if e1>e and e2>e and e3>e:
        continue
    else :
        break
print('\n solution : x = %0.3f, y = %0.3f and z = %0.3f\n'%(x1, y1, z1))
```

```
Enter tolerated error : 0.0001
         Iteration       x               y               z

         0               0.3333          1.3333          0.6667

         1               0.3333          1.6667          0.7778


 solution : x = 0.333, y = 1.667 and z = 0.778
```

III. Solve the following system of equation using the Gauss-Seidel method

$10x + y + z = 12,\ x + 10y + z = 12, x + y + 10z = 12.$

```
f1 = lambda x, y, z: (12-y-z)/10
f2 = lambda x, y, z: (12-x-z)/10
f3 = lambda x, y, z: (12-x-y)/10
x0, y0, z0 = 0, 0, 0
e = float(input('Enter tolerated error: '))
print('\t Iteration \t x \t\t y \t\t z \n')
for i in range (0, 25):
    x1 = f1(x0, y0, z0)
    y1 = f2(x1, y0, z0)
    z1 = f3(x1, y1, x0)
    print('\t %d \t\t %0.4f \t %0.4f \t %0.4f\n' %(i, x1, y1, z1))
    e1,e2,e3 = abs(x0-x1), abs(y0-y1), abs(z0-z1)
    x0,y0,z0 = x1,y1,z1
    if e1>e and e2>e and e3>e:
        continue
    else :
        break
print('\n Solution: x = %0.3f, y = %0.3f and z = %0.3f\n'% (x1, y1, z1))
```

```
Enter tolerated error: 0.0001
         Iteration       x               y               z

         0               1.2000          1.0800          0.9720

         1               0.9948          1.0033          1.0002

         2               0.9996          1.0000          1.0000

         3               1.0000          1.0000          1.0000


 Solution: x = 1.000, y = 1.000 and z = 1.000
```

2. Rayleigh's Power method

I. Compute numerically the largest eigenvalue of p=$\begin{bmatrix} 6 & -2 & 2 \\ -2 & 3 & -1 \\ 2 & -1 & 3 \end{bmatrix}$ by the power method.

```
from numpy import *
def normalize(x):
    lam = abs(x).max()
    x_n = x/lam
    return lam, x_n
x = matrix([[1], [1], [1]])
A = matrix([[6, -2, 2],[-2, 3, -1],[2, -1, 3]])
for i in range(10):
    x1 = dot(A,x)
    l, x = normalize(x1)
print('Eigenvalue:', l)
print('Eigenvector:', x)
```

```
Eigenvalue: 7.999988555930031
Eigenvector: [[ 1.        ]
 [-0.49999785]
 [ 0.50000072]]
```

II. .Compute numerically the largest eigenvalue of p= $\begin{bmatrix} 1 & 1 & 3 \\ 1 & 5 & 1 \\ 3 & 1 & 1 \end{bmatrix}$ **by power method.**

```
from numpy import *
def normalize(x):
    lam = abs(x).max()
    x_n = x/lam
    return lam, x_n
x = matrix([[1], [1], [1]])
A = matrix([[1, 1, 3], [1, 5, 1], [3, 1, 1]])
for i in range(10):
    x1 = dot(A,x)
    l, x = normalize(x1)
print('Eigenvalue:', l)
print('Eigenvector:', x)
```

```
Eigenvalue: 6.001465559355154
Eigenvector: [[0.5003663]
 [1.       ]
 [0.5003663]]
```

Exercise.

1 . Solve the following system of equation using Gauss- Seidel method

(i). $4x + y + z = 6,\ 2x + 5y - 2z = 5,\ x - 2y - 7z = -8.$

(ii). $27x + 6y - z = 85,\ 6x + 15y + 2z = 72,\ x + y + 54z = 110.$

2 . Solve the dominant eigenvalue of the matrix p= $\begin{bmatrix} 25 & 1 & 2 \\ 1 & 3 & 0 \\ 2 & 0 & -4 \end{bmatrix}$. Take $X_0 = (1, 0, 1)^T$.

3. Solve the dominant eigenvalue of the matrix p= $\begin{bmatrix} 6 & 1 & 2 \\ 1 & 10 & -1 \\ 2 & 1 & -4 \end{bmatrix}$. Take $X_0 = (1,1, 1)^T$.

4. Solve the dominant eigenvalue of the matrix p= $\begin{bmatrix} 5 & 1 & 1 \\ 1 & 3 & -1 \\ 2 & -1 & -4 \end{bmatrix}$. By power method.

LAB-5
Forward and Backward Interpolation.

1 . Newton's Forward Difference Formula

$$y(x) = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \frac{p(p-1)(p-2)(p-4)}{4!}\Delta^4 y_0 + \cdots$$

Where $p=(x-x_0)/h$ *and h is the subinterval length*

1. Generate forword interpolation table and hence calculate the interpolating value y(2) for the following

x	1	3	5	7	9
y	6	10	62	210	502

```
from numpy import *
def newton_forward(x, y, v):
    n = len(x)
    table = zeros((n, n))
    table[:,0] = y
    for j in range(1,n):
        for i in range(n - j):
            table[i][j] = table[i+1][j-1] - table[i][j-1]
    print("Forward Difference Table:")
    for i in range(n):
        print(f"{x[i]:0.2f}", end="\t")
        for j in range(n - i):
            print(f"{table[i][j]:0.2f}", end="\t")
        print()
    h = x[1] - x[0]
    p = (v - x[0]) / h
    result = y[0]
    p_term = 1
    for i in range(1, n):
        p_term *= (p - i + 1) / i
        result += p_term * table[0][i]
    return result
x = array([1, 3, 5, 7, 9])
y = array([6, 10, 62, 210, 502])
v = 2
inter_value = newton_forward(x, y, v)
print(f"\n Interpolated value at x = {v} is {inter_value:0.4f}")
```

```
Forward Difference Table:
1.00    6.00    4.00    48.00   48.00   0.00
3.00    10.00   52.00   96.00   48.00
5.00    62.00   148.00  144.00
7.00    210.00  292.00
9.00    502.00

 Interpolated value at x = 2 is 5.0000
```

II . Newton's Backward Difference Formula

$$y(x) = y_n + p\nabla y_n + \frac{p(p+1)}{2!}\nabla^2 y_n + \frac{p(p+1)(p+2)}{3!}\nabla^3 y_n + \frac{p(p+1)(p+2)(p+4)}{4!}\nabla^4 y_n + \cdots$$

Where $p=(x-x_n)/h$ and h is the subinterval length

1. Generate a backward interpolation table and hence calculate the interpolating value y(7) for the following

x	0	2	4	6	8	10
y	0	4	56	204	496	980

```
from numpy import *
def newton_backward(x, y, v):
    n = len(x)
    table = zeros((n, n))
    table[:,0] = y
    for j in range(1, n):
        for i in range(n - 1, j - 1, -1):
            table[i][j] = table[i][j-1] - table[i-1][j-1]
    print("Backward Difference Table:")
    for i in range(n):
        print(f"{x[i]:0.2f}", end="\t")
        for j in range(i + 1):
            print(f"{table[i][j]:0.2f}", end="\t")
        print()
    h = x[1] - x[0]
    p = (v - x[-1]) / h
    result = y[-1]
    p_term = 1
    for i in range(1, n):
        p_term *= (p + i - 1) / i
        result += p_term * table[n - 1][i]
    return result
x = array([0,2,4,6,8,10])
y = array([0, 4, 56, 204, 496, 980])
v1 = 7
inter_value = newton_backward(x, y, v1)
print(f"\n Interpolated value at x = {v1} is {inter_value:0.4f}")
```

```
Backward Difference Table:
0.00    0.00
2.00    4.00    4.00
4.00    56.00   52.00   48.00
6.00    204.00  148.00  96.00   48.00
8.00    496.00  292.00  144.00  48.00   0.00
10.00   980.00  484.00  192.00  48.00   0.00    0.00

 Interpolated value at x = 7 is 329.0000
```

Exercise.

1 . Using Newton's Backward interpolation method obtain y(160) for following data

x	100	150	200	250	300
y	10	13	15	17	18

2. Using Newton's Forward interpolation polynomial and calculate y(1) and y(10)

x	3	4	5	6	7	8	9
y	4.8	8.4	14.5	23.6	36.2	52.8	73.9

LAB -6

Lagrange's Interpolation.

1 . Using Lagranges interpolation Formula, find the value of y when x=10 for the following data

x	5	6	9	11
y=f(x)	12	13	14	16

```
def lagrange_interpolation(x, y, xp):
    n = len(x)
    yp = 0
    for i in range(n):
        Li = 1
        for j in range(n):
            if i != j:
                Li *= (xp - x[j]) / (x[i] - x[j])
        yp += y[i] * Li
    return yp
X = [5, 6, 9, 11]
Y = [12, 13, 14, 16]
xp = 10
yp = lagrange_interpolation(X, Y, xp)
print(f"interpolated value at x = {xp} is y = {yp}")
```

```
interpolated value at x = 10 is y = 14.666666666666666
```

2 . Obtain the Lagranges interpolation polynomial for the followingfor the following data and hence find f(5)

x	1	3	4	6
y=f(x)	-3	9	30	132

```
from sympy import *
x = symbols('x')
def lagrange_interpolation(X, Y, xp):
    n = len(X)
    poly = 0
    for i in range(n):
        Li = 1
        for j in range(n):
            if i!= j:
                Li *= (x - X[j]) / (X[i] - X[j])
        poly += Y[i] * Li
    poly1 = lambdify(x, poly)
    return expand(poly), poly1(xp)
X = [1, 3, 4, 6]
Y = [-3, 9, 30, 132]
xp = 5
p,v = lagrange_interpolation(X, Y, xp)
print("Lagrange Interpolating Polynomial:")
display(p)
print(f"Interpolating value at {xp} is {v}")
```

```
Lagrange Interpolating Polynomial:
```

$$x^3 - 3x^2 + 5x - 6$$

```
Interpolating value at 5 is 69.0
```

Exercise.

1 . if y(0)=-12, y(1)=0,y(3)=6 and y(4)=12, find the Lagrange's interpolation polynomial and estimate y(2)

2 . Using Lagrange's interpolation formula find f(4),given

x	1	3	4	6
y=f(x)	-3	9	30	132

LAB – 7
Numerical Integration

1 .Trapezoidal Method

I .Evaluate $\int_0^5 \frac{1}{1+x^2} dx$ by Trapezoidal rule , taken n=10.

```
def f(x):
    return 1/(1+x**2)
def trapezoidal(x0,xn,n):
    h = (xn-x0)/n
    s = f(x0)+f(xn)
    for i in range(1,n):
        k = x0+i*h
        s = s+2*f(k)
    return s*h/2
I = trapezoidal(0, 5, 10)
print("Integral value by Trapezoidal method is %0.4f" %I)
```

```
Integral value by Trapezoidal method is 1.3731
```

2 . Simpson's 1/3rd rule

I . Evaluate $\int_0^5 \frac{1}{1+x^2} dx$ by Simpson's 1/3rd rule , taken n=10.

```
def f(x):
    return 1/(1+x**2)
def simpsons13(x0,xn,n):
    h = (xn-x0)/n
    s = f(x0)+f(xn)
    for i in range(1,n):
        k = x0+i*h
        if i%2 == 0:
            s = s+2*f(k)
        else :
            s = s+4*f(k)
    return  s*h*(1/3)
I = simpsons13(0, 5, 10)
print("Integration value by Simpson's 1/3 rule is %0.5f" %I)
```

```
Integration value by Simpson's 1/3 rule is 1.37145
```

3 . Simpson's 3/8th rule

I . Evaluate $\int_0^6 \frac{1}{1+x^2} dx$ by Simpson's 3/8th rule, taken 6 subintervals.

```
def f(x):
    return 1/(1+x**2)
def simpsons38(x0,xn,n):
    h = (xn-x0)/n
    s = f(x0)+f(xn)
    for i in range(1,n):
        k = x0+i*h
        if i%3 == 0:
            s = s+2*f(k)
        else :
            s = s+3*f(k)
    return s*h*(3/8)
I = simpsons38(0, 6, 6)
print('Integration value by Simpsons 3/8 rule %3.5f'% I)
```

```
Integration value by Simpsons 3/8 rule 1.35708
```

Exercise.

1 . Evaluate the integral $\int_0^1 \frac{x^2}{1+x^3} dx$ by using Simpson's 1/3rd rule.

2 . Use Simpson's 3/8th rule to find $\int_0^{0.6} e^{-x^2} dx$ by having seven ordinates.

3 . Evaluate using the trapezoidal rule $\int_0^{\pi} sin^2 x \, dx$ Take n=6

LAB – 8

Taylor's Series Method, Modified Euler's method

1 . Taylor's Series Method

I . Solve $\frac{dy}{dx} - 2y = 3e^x$ with y(0)=0 using Taylor's series method at x=0.1 , 0.2, 0.3

```
from numpy import *
def deriv(x,y):
    D = zeros((4,1))
    D[0] = 2*y[0]+3*exp(x)
    D[1] = 4*y[0]+9*exp(x)
    D[2] = 8*y[0]+21*exp(x)
    D[3] = 16*y[0]+45*exp(x)
    return D
def taylor(deriv, x, y, xn, h):
    while x<xn:
        D = deriv(x,y)
        H = 1.0
        for i in range(4):
            H = H*h/(i+1)
            y = y+D[i]*H
        x = x+h
        print('y(%0.1f) = %0.4f'%(x,y))
taylor(deriv, 0.0, array([0.0]), 0.3, 0.1)
```

```
y(0.1) = 0.3487
y(0.2) = 0.8112
y(0.3) = 1.4167
```

II . Solve $y'+4y=x^2$ with the initial condition y(0)=1 using Taylor's series method at x=0.1, 0.2

```
from numpy import *
def deriv(x,y):
    D = zeros((4,1))
    D[0] = x**2-4*y
    D[1] = 2*x-4*x**2+16*y
    D[2] = 2-8*x+16*x**2-64*y
    D[3] = -8+32*x-64*x**2+256*y
    return D
def taylor(deriv, x, y, xn, h):
    while x<xn:
        D = deriv(x,y)
        H = 1.0
        for i in range(4):
            H = H*h/(i+1)
            y = y+D[i]*H
        x = x+h
        print('y(%0.1f) = %0.5f'%(x,y))
taylor(deriv, 0.0, array([1.0]), 0.2, 0.1)
```

```
y(0.1) = 0.67070
y(0.2) = 0.45164
```

2 . Modified Euler's method

I . Solve y′= -ky with y(0)=100 using modified Euler's method at x=100 , by taking h=25 (k=0.01)

```
def f(x,y):
    return -0.01*y
def eulerMod(x0, y0, xn, h):
    n = round((xn-x0)/h)
    for i in range(1,n+1):
        y1 = y0 + h*f(x0,y0)
        x1 = x0 + h
        print('y(%0.3f) = %0.4f'% (x1,y1))
        for j in range(1,4):
            y1 = y0+(h/2)*(f(x0,y0) + f(x1,y1))
            print('The',j,'th improved value of y(%0.3f)=%0.4f' %(x1,y1))
        x0, y0 = x1, y1
eulerMod(0, 100, 100, 25)
```

```
y(25.000) = 75.0000
The 1 th improved value of y(25.000)=78.1250
The 2 th improved value of y(25.000)=77.7344
The 3 th improved value of y(25.000)=77.7832
y(50.000) = 58.3374
The 1 th improved value of y(50.000)=60.7681
The 2 th improved value of y(50.000)=60.4643
The 3 th improved value of y(50.000)=60.5023
y(75.000) = 45.3767
The 1 th improved value of y(75.000)=47.2674
The 2 th improved value of y(75.000)=47.0311
The 3 th improved value of y(75.000)=47.0606
y(100.000) = 35.2955
The 1 th improved value of y(100.000)=36.7661
The 2 th improved value of y(100.000)=36.5823
The 3 th improved value of y(100.000)=36.6052
```

Exercise.

1 . Find y(0.1) by Taylor's Series expansion when $y' = x - y^2$, y(0)=1.

2 . Find y(0.2) by Taylor's Series expansion when $y' = x^2y - 1$, $y(0)=1$, $h=0.1$

3 . Evaluate by modified Euler's method $y' = ln(x+y)$, $y(0)=2$ at $x=0, 0.2, 0.8$.

4 . Solve by modified Euler's method $y' = x+y$, $y(0)=1$ $h=0.1, x=0, 0.1, 0.3$.

LAB – 9

Fourth order Runge- Kutta method.

1 . Apply the Runge-Kutta method to find the solution of $\frac{dy}{dx} = 1 + \frac{y}{x}$ at y(2) taking h=0.2 Given that y(1)=2.

```
def f(x, y):
    return 1+y/x
def RK(x0, y0, xn, h):
    n = round((xn-x0)/h)
    for i in range(1, n+1):
        k1 = h*f(x0, y0)
        k2 = h*f(x0+h/2, y0+k1/2)
        k3 = h*f(x0+h/2, y0+k2/2)
        k4 = h*f(x0+h, y0+k3)
        y1 = y0 + (1/6)*(k1+2*k2+2*k3+k4)
        x1 = x0 + h
        print('y(%0.1f) = %0.4f'%(x1,y1))
        x0, y0 = x1, y1
RK(1, 2, 2, 0.2)
```

```
y(1.2) = 2.6188
y(1.4) = 3.2710
y(1.6) = 3.9520
y(1.8) = 4.6580
y(2.0) = 5.3863
```

Exercise.

1 . Find *y(0.1)* by Runge-Kutta method when $y' = x - y^2$, y(0)=1.

2 . Find *y'= log(x+y). y(0)=2 at x=0,0.2,0.8 by* Runge-Kutta method.

LAB -10

Finding Gradient, Divergence and Curl

1. I) Find the gradient of $\phi = x^2y + 2xz - 4$

```
from sympy.physics.vector import *
from sympy import *
var('x,y,z')
v = ReferenceFrame('v')
F = v[0]**2*v[1]+2*v[0]*v[2]-4
G = gradient(F,v)
F = F.subs([(v[0], x),(v[1], y),(v[2], z)])
display("Given scalar function F=", F)
G = G.subs([(v[0], x),(v[1], y),(v[2], z)])
display("Grdient of F=", G)
```

'Given scalar function F='

$$x^2y + 2xz - 4$$

'Grdient of F='

$$(2xy + 2z)\hat{\mathbf{v}}_\mathbf{x} + x^2\hat{\mathbf{v}}_\mathbf{y} + 2x\hat{\mathbf{v}}_\mathbf{z}$$

II . Find the gradient of $\phi = x^2yz$

```
from sympy.physics.vector import *
from sympy import *
var('x,y,z')
v = ReferenceFrame('v')
F = v[0]**2*v[1]*v[2]
G = gradient(F,v)
F = F.subs([(v[0],x),(v[1],y),(v[2],z)])
display("Given scalar function F=",F)
G = G.subs([(v[0],x), (v[1],y),(v[2],z)])
display("Gradient of F=", G)
```

'Given scalar function F='

$$x^2yz$$

'Gradient of F='

$$2xyz\hat{\mathbf{v}}_\mathbf{x} + x^2z\hat{\mathbf{v}}_\mathbf{y} + x^2y\hat{\mathbf{v}}_\mathbf{z}$$

2 . Divergence of Vector Function

I . Find the divergence of $\vec{F} = x^2y\vec{i} + y^2z\vec{j} + z^2x\vec{k}$.

```
from sympy.physics.vector import *
from sympy import *
var('x,y,z')
v = ReferenceFrame('v')
F = v[0]**2*v[1]*v.x+v[2]*v[1]**2*v.y+v[2]**2*v[0]*v.z
D = divergence(F,v)
F = F.subs([(v[0],x),(v[1],y),(v[2],z)])
display("Given vector function F=", F)
D = D.subs([(v[0],x),(v[1],y),(v[2],z)])
display("Divergence of F=", D)
```

'Given vector function F='

$$x^2y\hat{\mathbf{v}}_\mathbf{x} + y^2z\hat{\mathbf{v}}_\mathbf{y} + xz^2\hat{\mathbf{v}}_\mathbf{z}$$

'Divergence of F='

$$2xy + 2xz + 2yz$$

II . Find the divergence of $\vec{F} = x^2yz\vec{i} + y^2zx\vec{j} + z^2xy\vec{k}$.

```
from sympy.physics.vector import *
from sympy import *
var('x,y,z')
v = ReferenceFrame('v')
F = v[0]**2*v[1]*v[2]*v.x + v[1]**2*v[2]*v[0]*v.y + v[2]**2*v[0]*v[1]*v.z
D = divergence(F,v)
F = F.subs([(v[0],x),(v[1],y),(v[2],z)])
display("Given vector function F=", F)
D = D.subs([(v[0],x),(v[1],y),(v[2],z)])
display("Divergence of F=", D)
```

'Given vector function F='

$$x^2yz\hat{\mathbf{v}}_\mathbf{x} + xy^2z\hat{\mathbf{v}}_\mathbf{y} + xyz^2\hat{\mathbf{v}}_\mathbf{z}$$

'Divergence of F='

$$6xyz$$

3.Curl of Vector Function

I . Find the Curl of $\vec{F} = xy^2\vec{i} + 2x^2yz\vec{j} - 3yz^2\vec{k}$.

```
from sympy.physics.vector import *
from sympy import *
var('x,y,z')
v = ReferenceFrame('v')
F = v[0]*v[1]**2*v.x+2*v[0]**2*v[1]*v[2]*v.y-3*v[1]*v[2]**2*v.z
C = curl(F,v)
F = F.subs([([0],x),(v[1],y),(v[2],z)])
display("Given vector point function is ", F)
C = C.subs([(v[0],x),(v[1],y),(v[2],z)])
display("curl of F=", C)
```

```
'Given vector point function is '
```

$$v_x y^2 \hat{\mathbf{v}}_\mathbf{x} + 2v_x^2 yz\hat{\mathbf{v}}_\mathbf{y} - 3yz^2\hat{\mathbf{v}}_\mathbf{z}$$

```
'curl of F='
```

$$(-2x^2y - 3z^2)\hat{\mathbf{v}}_\mathbf{x} + (4xyz - 2xy)\hat{\mathbf{v}}_\mathbf{z}$$

II . Find the Curl of $\vec{F} = x^2yz\vec{i} + y^2zx\vec{j} + z^2\,\overrightarrow{xyk}$.

```
from sympy.physics.vector import *
from sympy import *
var('x,y,z')
v = ReferenceFrame('v')
F = v[0]**2*v[1]*v[2]*v.x + v[1]**2*v[2]*v[0]*v.y + v[2]**2*v[0]*v[1]*v.z
C = curl(F,v)
F = F.subs([(v[0],x),(v[1],y),(v[2],z)])
display("Given vector point function is ", F)
C = C.subs([(v[0],x),(v[1],y),(v[2],z)])
display("curl of F=", C)
```

```
'Given vector point function is '
```

$$x^2yz\hat{\mathbf{v}}_\mathbf{x} + xy^2z\hat{\mathbf{v}}_\mathbf{y} + xyz^2\hat{\mathbf{v}}_\mathbf{z}$$

```
'curl of F='
```

$$(-xy^2 + xz^2)\hat{\mathbf{v}}_\mathbf{x} + (x^2y - yz^2)\hat{\mathbf{v}}_\mathbf{y} + (-x^2z + y^2z)\hat{\mathbf{v}}_\mathbf{z}$$

Exercise.

1 . If u=x+y+z, v=$x^2+y^2+z^2$, w=yz+zx+xy, find grad u, grad v, grad w.

2 . Evaluate div $\vec{F}$ and Curl $\vec{F}$ at nthe point (1,2,3), given that $\vec{F} = x^2yz\vec{\imath} + y^2zx\vec{\jmath} + z^2\,xy\vec{k}$.

3 . Prove that the vector $(yz - x^2)\vec{\imath} + (4y - z^2)\vec{\jmath} + (2xz - 4z)\vec{k}$ is solenoidal.

4. Find the vector normal to the surface $xy^3z^2 = 4$ at the point

5. If $\vec{R}$ =$x\vec{\imath} + y\vec{\jmath} + z\vec{k}$. Show that (i) $\nabla.\vec{R} = 3$ (ii) $\nabla \times \vec{R}$=$\vec{0}$